

PSTAT 10 Data Science Principles

Lecture 6: Dataframes

John Robin Inston 

johninston@ucsb.edu

University of California, Santa Barbara

August 11, 2026

Introduction

Review: Lecture 5

👉 Last lecture we explored some problem solving methods:

- **Algorithmic Thinking**
- Branching Logic
- Iterative Logic
- Recursive Logic
- Famous Algorithms:
 - Binary Search
 - Bubble Sort

👁️👁️ Outline: Lecture 6

👉 This lecture we will shift our focus to real data, specifically:

- Storing Data
- Factor Data



Figure 1: Data.

Dataframes

Real Data

So far the data we have considered has typically been very simplistic:

- Scalar values
- Defined vectors, matrices and arrays
- Basic lists without much nuance

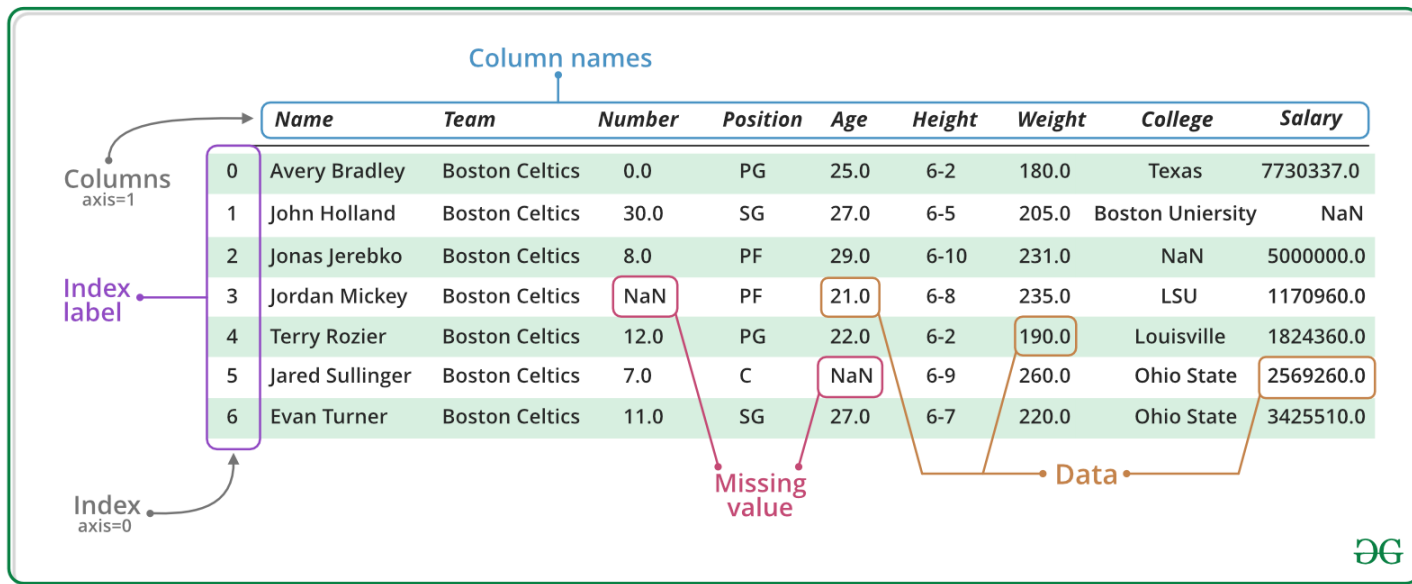
The only “real” data we have considered is the `iris` dataset.

```
1 # load data
2 data(iris)
3 # inspect data
4 iris[1:6, ]
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Dataframes

- When we load this data to our environment we see it is saved as a **dataframe** object.
- Dataframes are one of the most commonly used data types in R due to them helpfully combining the following useful attributes:
 - **Dimensional** (matrix structure)
 - **Varied datatypes**



The diagram illustrates a dataframe structure with the following components:

- Column names:** Name, Team, Number, Position, Age, Height, Weight, College, Salary.
- Index label:** Points to the first column (Name).
- Columns axis=1:** Points to the first column (Name).
- Index axis=0:** Points to the first row (0).
- Missing value:** Points to the 'NaN' value in the 'Number' column for the third row (index 3).
- Data:** Points to the 'Salary' column.

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston Uniersity	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Figure 2: Dataframes.

Tidyverse

A key package for **handling and manipulating dataframes** is `tidyverse`, which is a very large library consisting of several sub-libraries including:

- `tibble` — dataframe / **tibble** manipulation
- `tidyr` — **data cleaning** and processing
- `ggplot2` — **plotting**

Make sure you have the package installed using:

```
1 install.packages("tidyverse")
```

We will always load the `tidyverse` library for lectures going forward, however we will often not load the dataset libraries as they have a frustrating habit of redefining functions.

Some **dataset libraries** we shall be using include:

- `MASS` (`install.packages("MASS")`)
- `datasauRus` (`install.packages("datasauRus")`)

Loading Data

Loading data from a library

- To **load data** from a library (for example the `abbey` dataset from the `MASS` package) we use the `::` syntax.
- Whenever we load data we must use the **assignment operator** `<-` to store it in some suitably named object.

```
1 # load abbey dataset
2 abbey <- MASS::abbey
3 head(abbey)
```

```
[1] 5.2 6.5 6.9 7.0 7.0 7.0
```

 The `::` syntax allows us to avoid loading the entire `MASS` library.

`head()` Function

- Above we used the `head()` function.
 - This function is very useful for **initial inspection** of a dataset.
 - It returns the first n rows of a dataframe (default $n = 6$).



`data.frame()` Function

Defining dataframes in R

- We can also **manually construct** a dataframe from scratch using the `data.frame()` function.
- This function takes **one or more vectors** as input, treating each as a column.
- It also lets us control **row names**, **column names**, and how **data types** are handled.

The `data.frame()` Function

```
1 # dataframe function
2 data.frame(
3   ..., row.names = NULL, check.rows = FALSE,
4   check.names = TRUE, fix.empty.names = TRUE,
5   stringsAsFactors = FALSE
6 )
```

- We replace `...` with our data.
- Row and column names have defaults.
- There are lots of options and arguments — we will look through the main ones.

Constructing a Dataframe

The simplest way to **construct a data frame** is by defining each column as a vector, for example:

```
1 # define dataframe columns
2 booleans <- c(TRUE, FALSE, TRUE, TRUE)
3 characters <- c("dog", "seal", "cat", "cow")
4 numbers <- c(2, 5, 3.14, 1.26)
```

Then we can construct a dataframe using the `data.frame()` function:

```
1 df <- data.frame(
2   booleans,
3   characters,
4   numbers
5 )
6 df
```

	booleans	characters	numbers
1	TRUE	dog	2.00
2	FALSE	seal	5.00
3	TRUE	cat	3.14
4	TRUE	cow	1.26

Indexing

We can access a specific column of a dataframe using the `$` **syntax**:

```
1 df$characters
```

```
[1] "dog" "seal" "cat" "cow"
```

We can also use the **indexing syntax** we have seen before:

```
1 df[,2] # 2nd column
```

```
[1] "dog" "seal" "cat" "cow"
```

```
1 df[1,] # 1st row
```

	booleans	characters	numbers
1	TRUE	dog	2

Example — Constructing a Dataframe

Let's build a slightly more **complex** dataframe — this time with four columns of student records:

```
1 # define dataframe columns
2 name <- c("Priya", "Malik", "Sofia", "Owen")
3 age <- c(20, 22, 19, 21)
4 gpa <- c(3.8, 3.1, 3.95, 2.9)
5 enrolled <- c(TRUE, TRUE, FALSE, TRUE)
6 students <- data.frame(name, age, gpa, enrolled)
```

Printing `students`, we again see the object names become the column names:

```
1 students
```

	name	age	gpa	enrolled
1	Priya	20	3.80	TRUE
2	Malik	22	3.10	TRUE
3	Sofia	19	3.95	FALSE
4	Owen	21	2.90	TRUE

Loading Real Data

The main purpose of dataframes is to allow us to **manipulate real world data sets**. For example, consider the `cabbages` dataset.

```
1 # load from MASS
2 cabbages <- MASS::cabbages
```

We can use the following functions to make an **initial inspection** of the dataframe:

- `View()` — opens dataframe in new tab
- `head()` — (already seen) view the **first rows** of the dataframe
- `tail()` — view the **bottom rows** of the dataframe

Inspecting Data

```
1 # open in new tab (commented out for slides)
2 #View(cabbages)
3 # print first 3 rows
4 head(cabbages, 3)
5 # print last 3 rows
6 tail(cabbages, 3)
```

	Cult	Date	HeadWt	VitC
1	c39	d16	2.5	51
2	c39	d16	2.2	55
3	c39	d16	3.1	45

	Cult	Date	HeadWt	VitC
58	c52	d21	1.0	68
59	c52	d21	1.5	66
60	c52	d21	1.6	72

Dataframe Functions

Summarizing Dataframes

Often we wish to construct **summary statistics** for columns of the data frame (i.e. compute the mean, variance, minimum, maximum etc). To do so we use the `summary()` function.

```
1 summary(cabbages)
```

Cult	Date	HeadWt	VitC
c39:30	d16:20	Min. :1.000	Min. :41.00
c52:30	d20:20	1st Qu.:1.875	1st Qu.:50.75
	d21:20	Median :2.550	Median :56.00
		Mean :2.593	Mean :57.95
		3rd Qu.:3.125	3rd Qu.:66.25
		Max. :4.300	Max. :84.00

Structure of Dataframes

Additionally, we may wish to understand the **structure** of a dataframe, i.e. the **dimensions**, what **datatype** is in each column, etc.

We can do so using the `str()` function.

```
1 str(cabbages)
```

```
'data.frame': 60 obs. of 4 variables:
 $ Cult : Factor w/ 2 levels "c39","c52": 1 1 1 1 1 1 1 1 1 ...
 $ Date : Factor w/ 3 levels "d16","d20","d21": 1 1 1 1 1 1 1 1 1 ...
 $ HeadWt: num 2.5 2.2 3.1 4.3 2.5 4.3 3.8 4.3 1.7 3.1 ...
 $ VitC : int 51 55 45 42 53 50 50 52 56 49 ...
```

Indexing Dataframes

Numerical Indexing

We **index a dataframe** in the same way as a matrix, by using an **ordered pair** (i, j) where i is the row number and j is the column number:

```
1 # 1st row, 1st column
2 cabbages[1,1]
3 # 3rd row, columns 1 to 3
4 cabbages[3, 1:3]
5 # 1st and 2nd rows, 1st and 2nd columns
6 cabbages[1:2, 1:2]
```

```
[1] c39
Levels: c39 c52
```

```
   Cult Date HeadWt
3  c39   d16    3.1
```

```
   Cult Date
1  c39   d16
2  c39   d16
```

Column Name Indexing

We can also index a dataframe by **column name**:

```
1 # 1st row, column "HeadWt"
2 names(cabbages)
3 cabbages[1, "HeadWt"]
```

```
[1] "Cult" "Date" "HeadWt" "VitC"
```

```
[1] 2.5
```


Slicing Dataframes

Slicing Dataframes

We can **return entire rows / columns** by leaving the **column / row index blank** respectively:

```
1 # return the 3rd row
2 cabbages[3,]
3 # return the 4th column
4 cabbages[,4]
```

```
Cult Date HeadWt VitC
3 c39 d16 3.1 45
```

```
[1] 51 55 45 42 53 50 50 52 56 49 65 52 41 51 41 45 51 45 61 42 54
59 66 54 45
[26] 49 49 55 49 68 58 55 67 61 67 68 58 63 56 72 52 70 57 58 47 56
72 63 54 60
[51] 78 75 70 84 71 72 62 68 66 72
```

Dollar Sign Indexing

We can also use the **dollar sign**  **syntax** to return a column by name:

```
1 head(cabbages$HeadWt)
```

```
[1] 2.5 2.2 3.1 4.3 2.5 4.3
```

Dataframe Behavior

Functions on Dataframes

Once we have indexed rows / columns they **behave as vectors**, and we can perform calculations / apply functions as we have been doing in previous lectures.

```
1 # mean of HeadWt column
2 mean(cabbages$HeadWt)
3 # length of HeadWt column
4 length(cabbages$HeadWt)
```

```
[1] 2.593333
```

```
[1] 60
```

Some dataframe-specific functions we will explore in upcoming lectures include:

- `order()`
- `subset()`
- `apply()`

Exploratory Data Analysis

Exploratory Data Analysis

Real world data is messy, and so whenever we obtain new data / start some data analysis we always begin with **exploratory data analysis (EDA)**.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of analyzing data sets to summarize their main characteristics, often using visual methods.

- It is a crucial step in any data analysis process, as it helps to:
 - Uncover patterns, spot anomalies, test hypotheses, and check assumptions with the help of summary statistics and graphical representations.

We aim to investigate:

- How **spread out** is our data?
- Are any variables **correlated**?
- Do any values look wildly out of place? (**outliers**)
- Is any data **missing**?



Data Visualization Tools

Pictures paint a thousand words! Often the most intuitive way to explore data is using **visualizations**.

Boxplots

- Visually show the minimum, maximum, mean, upper and lower quartiles of the data.

Density Plots

- Similar to histograms but visualize an approximate density.

Histograms

- Visualize the empirical distribution of the data by grouping into bins and showing the number of counts per bin.

Correlation Plots

- A scatter plot of one variable against the other with a line of best fit, measuring the linear relationship between the two.

 We will explore each of these visualization tools in more detail in the following slides.



Example — Data Visualization

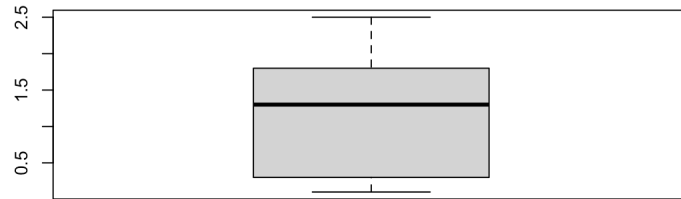
We produce the following series of plots using the `iris` dataset.

```
1 data("iris")
2 par(mfrow=c(2,2)) # plot a 2x2 grid of plots
3 # boxplot
4 boxplot(iris$Petal.Width, main = "Boxplot of Petal Width")
5 # histogram
6 hist(iris$Petal.Width, main = "Histogram of Petal Width",
7      xlab = "Petal Width")
8 # density plot
9 plot(density(iris$Petal.Width), main = "Kernel Density of Petal Width")
10 # correlation plot
11 plot(iris$Petal.Width, iris$Petal.Length,
12      main = "Correlation of Petal Length and Petal Width",
13      ylab = "Petal Length", xlab = "Petal Width")
14 abline(lm(iris$Petal.Length ~ iris$Petal.Width), col = "red", lty = "dashed")
15 text(paste("Correlation:", round(cor(iris$Petal.Width, iris$Petal.Length), 2)),
16      x = 1.5, y = 2)
```

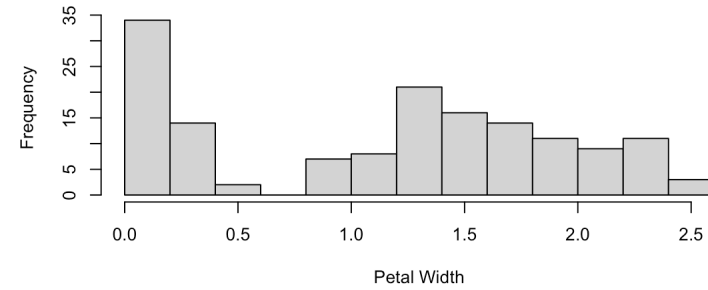


Example — Data Visualization

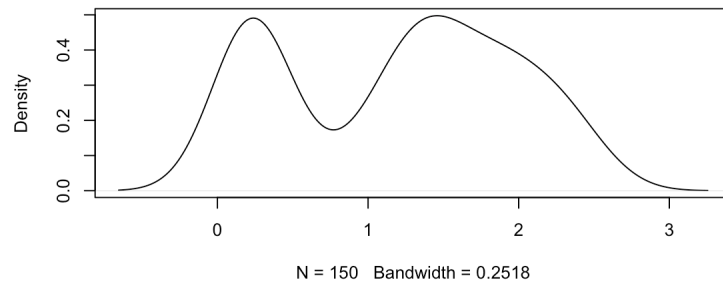
Boxplot of Petal Width



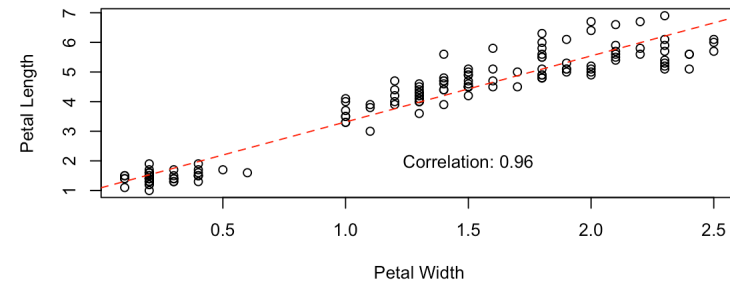
Histogram of Petal Width



Kernel Density of Petal Width



Correlation of Petal Length and Petal Width



Exploratory Data Analysis Plots

Summary Statistics — Mean & Standard Deviation

Mean

Recall that the **arithmetic mean** of a sample $\vec{x} := (x_1, \dots, x_n)$ is defined by

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}.$$

Intuitively, this is giving the average value of the sample (average location of all the points).

Standard Deviation

The **standard deviation** of a vector $\vec{x} := (x_1, \dots, x_n)$ is defined by

$$s_x = sd(\vec{x}) = \left(\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \right)^{1/2}.$$

Intuitively, this is computing the average distance of all points in the sample from the mean.



Summary Statistics — Covariance & Correlation

Covariance

Given two variables (vectors) $\vec{x} = (x_1, \dots, x_n)$ and $\vec{y} = (y_1, \dots, y_n)$ the **covariance** of $\vec{x}$ and $\vec{y}$ is given by

$$\text{Cov}(\vec{x}, \vec{y}) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

Intuitively, this measures whether two variables tend to move together.

 **Covariance** is hard to interpret on its own since its scale depends on the units of `x` and `y` — **correlation** fixes this by rescaling covariance to always fall between -1 and 1 .

Correlation

$$\text{Corr}(\vec{x}, \vec{y}) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \times \sum_{i=1}^n (y_i - \bar{y})^2}}.$$

Correlation is a statistical measure that expresses the extent to which two variables are linearly related

(meaning they change together at a constant rate).



Interpreting Correlation

Correlation takes a value on the interval $[-1, 1]$ where:

- -1 indicates **perfect negative correlation** (as one variable increases/decreases the other decreases/increases)
- 0 indicates **no correlation**
- 1 indicates **perfect positive correlation** (as one variable increases/decreases the other increases/decreases)

📌 We typically only note correlation values greater than 0.5 — anything lower is often **not considered significant**.

📌 A quick, informal note on **statistical significance**: seeing a correlation number doesn't automatically mean the relationship is "real." Significance is really just asking, "could I have gotten a number this big just by chance, from a small or noisy sample?" With very little data, even a large correlation can be a fluke; with lots of data, even a small one can be trustworthy. We won't cover the formal test for this in the course — just keep the question in the back of your mind whenever you see a correlation!

Statistics in the Background

These statistics can be time consuming to compute, and fortunately R has built-in functions for all three:

- `mean()` — computes the mean
- `sd()` — computes the standard deviation
- `cor()` — computes the correlation

For example, considering `Petal.Length` and `Petal.Width` from the `iris` dataset:

```
1 mean(iris$Petal.Length)
2 sd(iris$Petal.Length)
3 cor(iris$Petal.Length, iris$Petal.Width)
```

```
[1] 3.758
```

```
[1] 1.765298
```

```
[1] 0.9628654
```

Exercise — Why Visualize?

Evaluate the following dataset:

```
1 # load data
2 dino <- datasauRus::datasaurus_dozen[1:142, ]
3 # initial inspection
4 head(dino)
```

	dataset	x	y
1	dino	55.3846	97.1795
2	dino	51.5385	96.0256
3	dino	46.1538	94.4872
4	dino	42.8205	91.4103
5	dino	40.7692	88.3333
6	dino	38.7179	84.8718

1. Using the functions from the previous slide, compute the **mean**, **standard deviation** and **correlation** of the `x` and `y` columns.
2. Why might this dataset be a good example of why data visualization is important?

02:00



Solution — Why Visualize?

```
1 mean(dino$x)
2 sd(dino$x)
3 cor(dino$x, dino$y)
```

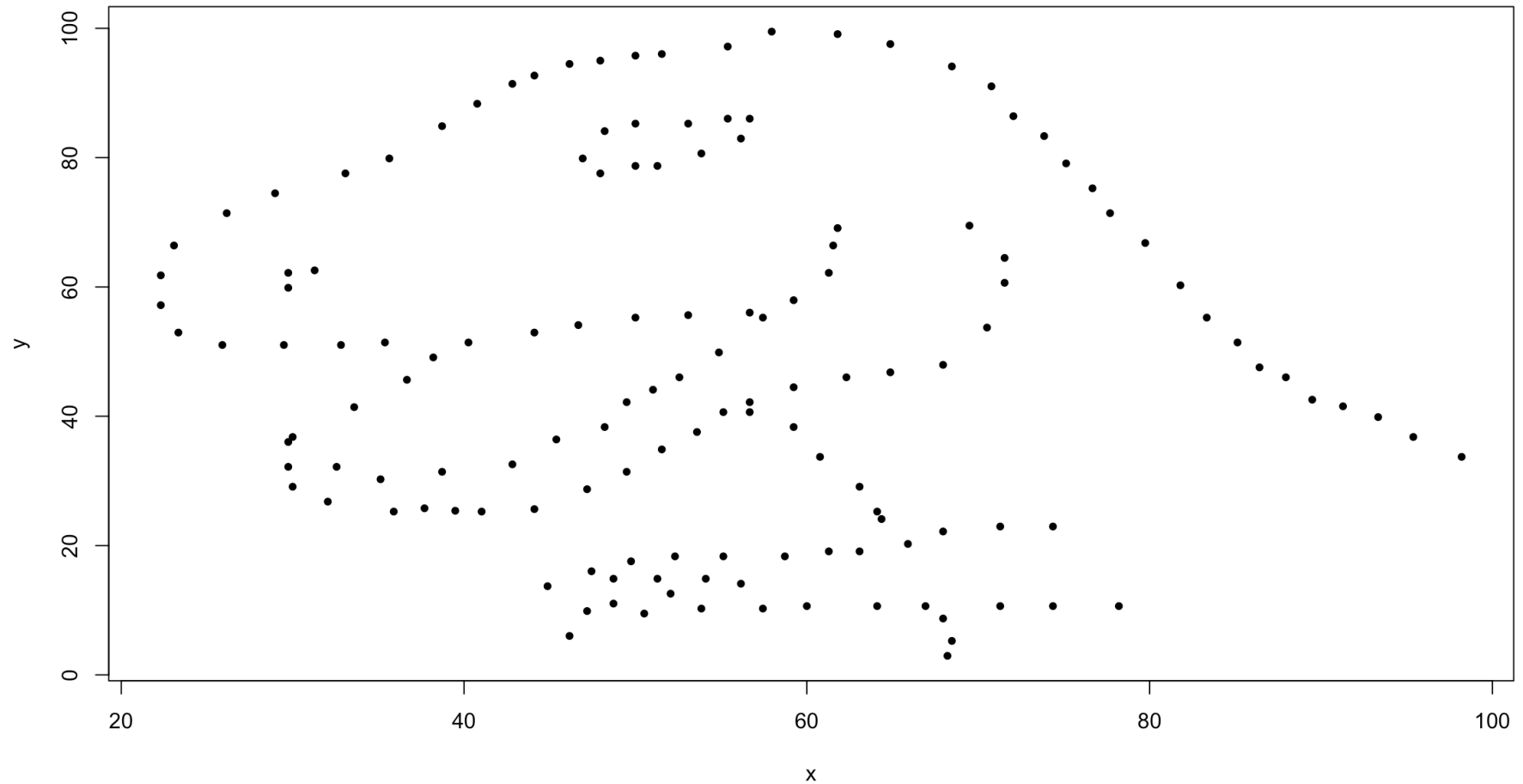
```
[1] 54.26327
```

```
[1] 16.76514
```

```
[1] -0.06447185
```

```
1 plot(dino$x, dino$y, pch = 20, main="Datasaurus Dozen", xlab = "x", ylab = "y")
```

Datasaurus Dozen



 The summary statistics of `dino` look completely unremarkable — it's only the **visualization** that reveals the hidden structure!

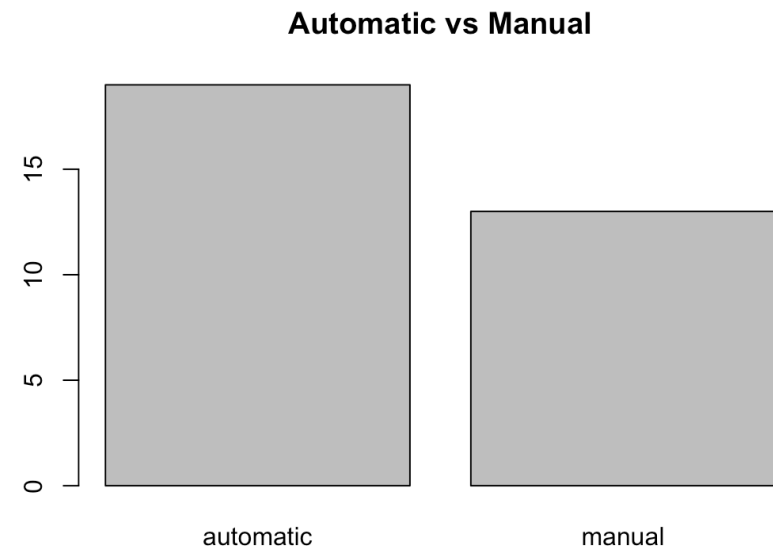
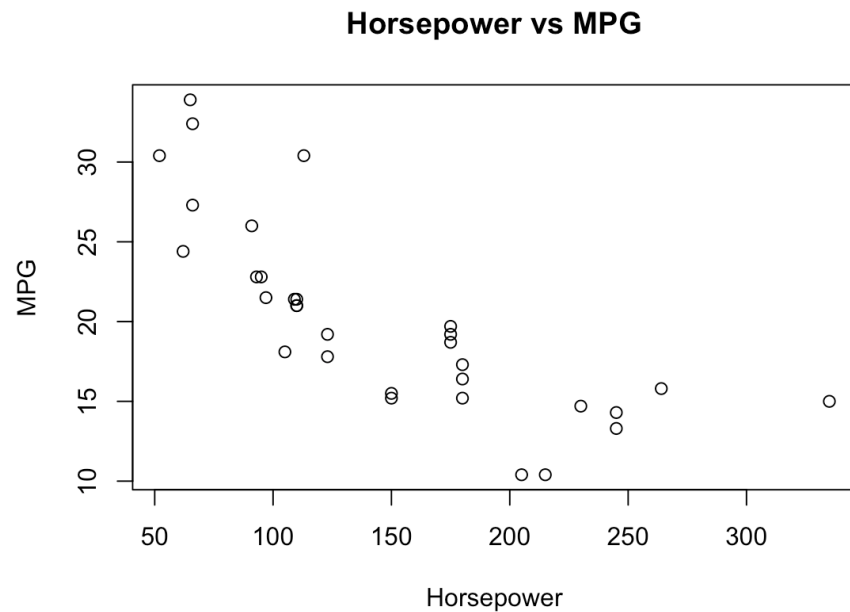
Categorical vs Quantitative Data

Categorical vs Quantitative

Dataframe data can roughly be broken down into the following two categories:

1. **Quantitative** (numerical) data; and
2. **Categorical** (factor, non-numerical) data.

We need to use **different visualization techniques** for each category:



Quantitative vs Categorical Visualization

Quantitative vs Categorical — Details

Quantitative Data Types

Quantitative data separates into two types:

- **Continuous** (age, height, weight, etc.)
 - Can take any value in a range (e.g. 1.5, 1.51, 1.511, etc.)
- **Discrete** (usually counts, e.g. shoe size, number of students)
 - Can only take specific values (e.g. 1, 2, 3, etc.)
 - Mathematically we say the set is **countable**.

Categorical Data

Any non-numerical data is considered categorical:

- Numbers can be considered categorical.
- Classification into **groups** (1, 2, 3, etc.); identification numbers.
- No ordering is necessary.

Factors in R

- R has excellent functionality for dealing with categorical data, using a specific datatype called `factors`:
 - The distinct categories of the data are called **levels**.
 - Each element of the data is assigned to the correct level.

```
1 # define categorical data
2 x <- c("red", "blue", "green", "blue", "green", "blue", "red", "yellow")
3 # create factor data
4 x <- factor(x)
5 # summarize the data
6 summary(x)
7 levels(x)
```

blue	green	red	yellow
3	2	2	1

```
[1] "blue" "green" "red" "yellow"
```



Visualization Tools by Data Type

We have to adjust our exploratory data analysis (EDA) based on whether we are considering categorical (factor) data or quantitative (numerical) data.

Categorical Plots

- Bar charts
 - Bar height/length shows the **count** in each category.
- Pie charts
 - A circle sliced up to show each category's **share of the whole**.
- Mosaic plots
 - Tile sizes show how two **categorical variables** relate to each other.

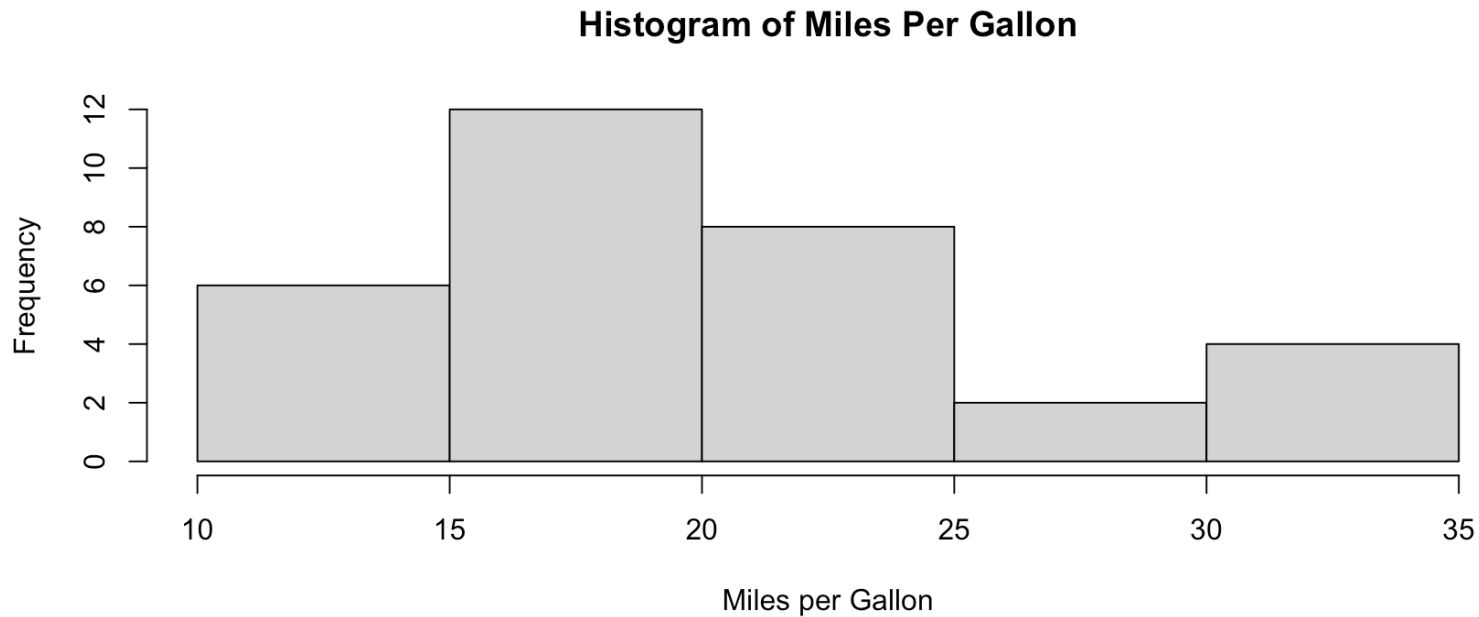
Quantitative Plots

- Histograms
 - Bars show how values are **distributed** across bins.
- Box plots
 - Summarizes **spread** via quartiles, median, and outliers.
- Scatter plots
 - Points show the **relationship** between two numeric variables.
- Line plots
 - Connected points track how a value **changes in order** (e.g. over time).

Histograms

Histograms split the range of the continuous data into **discrete intervals** (bins) and visualize the **count of observations in each bin**.

```
1 hist(mtcars$mpg, main = "Histogram of Miles Per Gallon", xlab = "Miles per Gallon", breaks=8)
```



Summary

Topics Covered

 Today we looked into:

- Real Data
 - Loading Data
 - Summarizing Data
 - Visualizing Data
- Exploratory Data Analysis
 - Quantitative vs Categorical
 - Factor Data
 - Histograms



Next Class



Next class we will look more carefully at dataframes, their problems and the solution, namely:

- Tibbles
- Tidyverse Package